CORE ATTENDEE EDITION: Start with 00_START_HERE.md at the pack root. Only the core two-day route is supplied. Optional publishing, dummy website, resource creation, identity/networking and advanced Azure tracker extensions mentioned below are not assigned or included. Use your own extracted path where a command shows C:\YOUR-ATTENDEE-PACK. Do not copy instructor settings.

Service Bus Explorer - a short standalone queue lab

LBS | Attendee edition | Fictional data only | Updated 28 October 2026

Before you start

Expected duration: 35 minutes: 15:00-15:35

Prerequisites: Existing assigned namespace and dedicated empty disposable queue; prepared by the owner. No active consumer on this queue.

Required permissions: Appropriate portal discovery access plus approved Azure Service Bus Data Sender and Data Receiver permissions for the assigned queue/namespace as required by the portal route. Contributor alone is not data permission.

Resources created: None on the core route. Namespace/queue creation is optional, separately approved.

Cost considerations: Tier/capacity and operations can charge. Send one tiny fictional message; avoid loops, large bodies and tier changes.

Safety: Use the assigned tenant, subscription and resource only. Do not change shared networking, roles, tiers or deployment settings. No real documents, secrets or keyed URLs in evidence.

Why a queue?

A caller can hand off work without waiting for processing to finish. A queue decouples producer and consumer; it is not a database of final business outcomes.

Example: submit a protected reference to a document, not the document or customer details. Our portal message does not trigger the local tracker or the Day 1 Function.

Time split: concepts 7; tutor demonstration 8; practice 12; verification/cost/cleanup 8 minutes.

Portal send, peek and receive

1. Open the assigned Service Bus namespace > Entities > Queues > your disposable queue > Service Bus Explorer. Confirm no real/shared workload or running consumer. Choose the supported Entra-authenticated mode if the interface offers authentication selection.

Expected result: The exact isolated queue is visible. If the route requires unapproved keys or permission, stop and use the tutor demonstration.

2. Use Send to submit one fictional reference such as LBS-DEMO-QUEUE-your-suffix-001 with a fresh MessageId if the editor exposes it. Record that ID privately without credentials.

Expected result: The send succeeds. Queue counts may refresh later; a successful send is not proof of business processing.

3. Choose Peek and inspect your message. Do not select or settle anyone else's message.

Expected result: The body/reference is visible; Peek does not remove or complete the message.

4. On this isolated queue only, choose Receive. If the UI offers PeekLock, receive with that mode and explicitly Complete your message. If it offers only ReceiveAndDelete, obtain approval for destructive receive first; do not expect a later Complete action.

Expected result: PeekLock plus Complete settles the received message; ReceiveAndDelete removes it on receive. Counts update eventually and other consumers would change the result, hence the isolated queue prerequisite.

5. Refresh or Peek again and verify your settled message is absent. Discuss retries and the dead-letter queue without forcing failures.

Expected result: You distinguish Peek, Receive and settlement. No Function trigger, replay or integration was created.

Troubleshooting - symptom, checks and safe recovery

Symptom	Exact checks / recovery
Namespace not globally available during optional creation	Choose an owner-approved unique suffix after checking policy; do not create alternate namespaces on the core route.
Send/receive denied	Check actual identity, data role, scope, network route and approved propagation delay. Do not use RootManageSharedAccessKey.
Message disappeared before Peek	Stop and check for a competing consumer or TTL. Do not repeatedly send into a shared workload.
Lock expired / Complete fails	On the disposable queue, receive your still-available message again if approved and complete within the lock. Delivery can repeat; real consumers need idempotent business effects.

Knowledge check - before and after

Before: does sending a message prove the business task completed?

After: what is the difference between Peek and ReceiveAndDelete? Why can a consumer see a message again? What should a dead-letter replay require before it is safe?

Cleanup

Only settle your one authorised test message. Never purge a shared queue.

Leave namespace/queue resources in place unless their owner approves deletion. Idle capacity can still charge.

Identity-based triggers, .NET senders, dead-letter replay and fault injection are optional separately prepared extensions, not this 35-minute core route.

Your evidence

Record	Your result
Mode: live / prepared query / offline review	
Scope, time range and source	
Expected / actual result	
Passed / Blocked / Not run and why	
Cost/security trade-off	
Follow-up owner / cleanup	